

BDI Gold Miner - Formal Specification

Theoretical Foundation: Wooldridge (2009) *An Introduction to MultiAgent Systems* §2.3–2.4 and Rao & Georgeff (1995) *BDI Agents: From Theory to Practice*

1. Overview

A spatial **BDI Gold Miner / Market Trader** model. Agents navigate a torus grid to forage for regenerating gold deposits, manage biological homeostasis, and trade at market nodes under fluctuating supply-and-demand economics.

Property	Why it matters
Multi-tick plans (navigate, mine, sell)	Exposes the missing Task primitive (Mesa Discussion #2526)
Four-phase deliberation cycle	Maps to Wooldridge's equations; cyclomatic complexity is quantifiable
Commitment strategy	No declarative strategy exists in Mesa - requires ad-hoc code
Decentralised market	Synchronous tick prevents true asynchronous price discovery

2. Variable Definitions

Agent State

Symbol	Name	Python attribute	Range	Description
$E(t)$	Energy	<code>self.energy</code>	$[0, E_{\max}]$	Biological reserve; depletes each tick
$G(t)$	Gold held	<code>self.gold_held</code>	$[0, \kappa]$	Units of gold in inventory
$W(t)$	Wealth	<code>self.wealth</code>	$[0, \infty)$	Cumulative revenue from sales
$\mathbf{p}(t)$	Position	<code>self.cell</code>	Grid $[0, W) \times [0, L)$	Current cell
$\mathbf{u}(t)$	Intention	<code>self.current_intention</code>	Intention enum	Active cognitive commitment

Intention States

Constant	Meaning
IDLE	No active intention; agent wanders
NAVIGATE_MINE	Moving toward best known mine
MINING	At a mine, extracting gold one unit per tick
NAVIGATE_MARKET	Moving toward best known market
SELLING	At a market, executing trade

Parameters

Symbol	Parameter	Default	Description
κ	Carry capacity	5	Maximum gold units held
v	Vision radius	5	Moore neighbourhood search radius
$E_{\max}$	Max energy	100	Upper clamp on energy
κ_M	Mine capacity	5	Units of gold per mine
T_M	Mine regen time	20	Ticks until depleted mine refills
p_0	Base price	10.0	Gold price at equilibrium

3. Formal Equations

3.1 The BDI Deliberation Cycle (Wooldridge, 2009 §2.3)

Every tick the agent executes four cognitive phases in sequence:

```
B(t+1) = brf(B(t), P(t)) (1)
D(t+1) = generate_options(B(t+1), I(t)) (2)
I(t+1) = filter(B(t+1), D(t+1), I(t)) (3)
         execute(plan(I(t+1))) (4)
```

All four phases are called sequentially inside `BDIMiner.step()`:

```
self._update_beliefs() # Eq. 1
self._update_desires() # Eq. 2
self._filter_intentions() # Eq. 3
if self.current_intention == Intention.NAVIGATE_MINE:
    self._execute_navigate_mine() # Eq. 4
elif self.current_intention == Intention.MINING:
    ...
```

Pain point [P5]: All four phases are entangled in one method. Testing belief revision in isolation requires constructing a full agent in a full model.

3.2 Biological Homeostasis

$$E(t+1) = \max(0, E(t) - 1.0) \quad (5)$$

$$S(t) = \begin{cases} 1 & ; \text{ if } E(t) > 0 \\ 0 & ; \text{ otherwise (agent removed) } \end{cases} \quad (6)$$

Pain point [P1]: Every decaying variable requires an explicit $\max(0, x - \lambda)$ assignment.

Pain point [P2]: The survival check is polled every tick even when energy is at 99.

3.3 Spatial Pathfinding

Observable set with Moore neighbourhood, radius v :

$$P_{\text{obs}} = \{ p' : \max(|x'-x|, |y'-y|) \leq v \} \quad (7)$$

Greedy one-step movement toward target p^* :

$$p(t+1) = \operatorname{argmin} \|q - p^*\|_2, \quad q \in N_1(p) \quad (8)$$

3.4 Belief Revision

Best mine — highest gold-per-distance ratio:

$$m^* = \operatorname{argmax} G_m / (1 + \|p_m - p\|_2), \quad m \in B_{\text{mines}} \quad (9)$$

Best market — highest price-per-distance ratio:

$$k^* = \operatorname{argmax} p_k / (1 + \|p_k - p\|_2), \quad k \in B_{\text{markets}} \quad (10)$$

Stale mine beliefs (depleted since last perception) are pruned each tick via a dict comprehension.

3.5 Option Generation — Soft Priorities

Desires are continuous utility scores that change smoothly with agent state:

$$\pi_{\text{survive}}(t) = \begin{cases} 200 & ; \text{ if } E(t) < 30 \\ 0 & ; \text{ otherwise } \end{cases} \quad (11)$$

$$\pi_{\text{sell}}(t) = 100 \cdot G(t) / \kappa \quad (12)$$

$$\pi_{\text{mine}}(t) = \begin{cases} 60 & ; \text{ if } G(t) < \kappa \text{ and } m^* \neq \emptyset \\ 0 & ; \text{ otherwise } \end{cases} \quad (13)$$

Pain point [P3]: The constants 200, 100, 60 are inline literals in

`_update_desires()`. Changing relative urgency requires editing scattered code with no compiler check.

3.6 Filtering — Blind Commitment Heuristic

The agent commits to its current intention unless a strictly higher-utility desire appears:

```
ι(t+1) = 
$$\begin{cases} \iota(t) & \text{and } \iota(t) \neq \text{IDLE} & ; \text{ if } \max_d \pi_d(t) \leq \pi_{\iota(t)}(t) \\ \text{new intention} & ; \text{ otherwise} \end{cases}$$
 (14)
```

Pain point [P7]: This is an ad-hoc **Blind Commitment** strategy. Wooldridge (2009 §2.4) defines three formal strategies:

- **Bold** — never interrupt a running intention
- **Cautious** — interrupt whenever any higher-priority desire appears
- **Open-minded** — re-evaluate on each new perception

None can be selected declaratively in vanilla Mesa. Changing strategy requires rewriting `_filter_intentions()` entirely.

3.7 Plan Execution — Tick Amnesia

Because Mesa is built on an event scheduling engine, durative tasks like navigation must be modelled as per-tick arrival checks:

```
p(t) = p_m* 
$$\begin{cases} \text{MINING} & ; \text{ if } \iota(t) = \text{NAVIGATE\_MINE} \text{ and} \\ \text{NAVIGATE\_MINE (move one step)} & ; \text{ otherwise} \end{cases}$$
 ι(t+1) = (15)
```

Pain point [P6] — GAMA comparison:

```
// GAMA: navigate to mine (1 line, duration handled by engine)
do goto target: best_mine.location;

# Mesa : navigate to mine (~8 lines per destination, duplicated)
def _execute_navigate_mine(self) -> None:
    if self.beliefs["best_mine"] is None:
        self._adopt_intention(Intention.IDLE)
        return
    if self.cell is self.target_cell:
        self.current_intention = Intention.MINING
        return
    self._move_toward(self.target_cell)
```

`_execute_navigate_market()` is **structurally identical** — duplicated boilerplate.

3.8 Market Economics

Revenue from selling G units (Eq. 16):

$$W(t+1) = W(t) + G(t) \cdot p_k^*(t) \quad (16)$$

Supply penalty (Eq. 17):

$$p_k_after = \max(1.0, p_0 - G(t) \cdot 0.1) \quad (17)$$

Price recovery each step (Eq. 18):

$$p_k(t+1) = \min(p_0, p_k(t) + 0.5) \quad (18)$$

Selling is the **only zero-duration action** — completes in one tick, no per-tick arrival check needed. Not every action requires a duration primitive.

3.9 Death Accounting

$$deaths(t) = N_start(t) - N_end(t) \quad (19)$$

4. Environment Dynamics

`GoldMine.mine_one()` guards with `if self.gold > 0` before scheduling regeneration — idempotent, safe to call multiple times per step.

`Market.reset_price()` is called once per step before any miner activates, so all miners see a consistent price baseline. The synchronous step order prevents true asynchronous price discovery — Phase 3 will test whether `TaskManager`'s continuous-time execution changes macro-level price equilibria.

5. Documented Pain Points

#	Pain Point	Code location	Framework solution (Phase 3)
P1	Manual decay — one <code>max(0, x - λ)</code> line per variable	<code>BDIMiner.step()</code> line 1	<code>BehavioralState (decay_rate=λ)</code>
P2	Survival check polled every tick	<code>BDIMiner.step()</code> line 2	<code>BehavioralState (thresholds={"dead": 0})</code>
P3	Desire priorities are inline literals	<code>_update_desires()</code>	<code>@rule(priority=RulePriority.URGENT)</code>
P5	All four BDI phases entangled in <code>step()</code>	All of <code>BDIMiner.step()</code>	Separate <code>@rule</code> methods + <code>DecisionSystem</code>
P6	Tick amnesia — navigation is a per-tick arrival check; <code>_execute_navigate_market()</code>	<code>_execute_navigate_mine()</code> , <code>_execute_navigate_market()</code>	<code>Task(action=self._move_toward, until=arrived)</code>

#	Pain Point	Code location	Framework solution (Phase 3)
	te_mine and _execute_navigate_market are 100% duplicated		
P7	Ad-hoc commitment — no declarative bold/cautious/open-minded strategy; changing it requires rewriting _filter_intentions()	_filter_intentions()	DecisionSystem(commitment="cautious")

[P6] and [P7] are unique to BDI — they do not appear in the Needs-Based baseline.

6. References

- Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Wiley. §2.3–2.4.
- Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. *ICMAS*, 312–319.
- GAMA BDI Tutorial: <https://gama-platform.org/wiki/BDIAgents>
- Mesa Discussion [#2526: Tasks](#)
- Mesa Discussion [#2529: Continuous States](#)
- Mesa Discussion [#2538: Behavioral Framework](#)
- Mesa Discussion [#3304: Actions — Event-driven Agent Behavior](#)